



TRAVAUX DIRIGÉS

Algorithmique et Programmation

Problèmes d'optimisation, de graphes et de combinatoire

Ce document regroupe une sélection de problèmes algorithmiques classiques issus de compétitions de programmation. Pour chaque exercice, une formulation détaillée est donnée, suivie de pistes de résolution.

1 Produit scalaire minimal

Exercice 1 — Produit scalaire minimal

On dispose de deux vecteurs $\mathbf{v}_1 = (x_1, x_2, \dots, x_n)$ et $\mathbf{v}_2 = (y_1, y_2, \dots, y_n)$. Le **produit scalaire** de ces deux vecteurs est défini par :

$$\mathbf{v}_1 \cdot \mathbf{v}_2 = \sum_{i=1}^n x_i y_i = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$$

On vous autorise à **permuter** les coordonnées de chaque vecteur comme vous le souhaitez. Trouvez deux permutations σ et τ des indices $\{1, \dots, n\}$ telles que le produit scalaire :

$$\sum_{i=1}^n x_{\sigma(i)} y_{\tau(i)}$$

soit le plus **petit** possible. Affichez cette valeur minimale.

Format d'entrée / sortie

Entrée : La première ligne contient T , le nombre de cas de test. Chaque cas contient un entier n ($1 \leq n \leq 800$), suivi de deux lignes de n entiers chacune (les coordonnées de $\mathbf{v}_1$ puis $\mathbf{v}_2$). Les valeurs sont comprises entre $-100\,000$ et $100\,000$.

Sortie : Pour chaque cas, afficher **Case #i :** V où V est le produit scalaire minimal.

Exemple :

$\mathbf{v}_1 = (1, 3, -5)$, $\mathbf{v}_2 = (-2, 4, 1)$.

Produit scalaire minimal : $1 \times 4 + 3 \times 1 + (-5) \times (-2) = 4 + 3 + 10 = 17$

Hmm, essayons: $(-5) \times 4 + 1 \times 1 + 3 \times (-2) = -20 + 1 - 6 = -25$. C'est mieux!

- **Observation clé :** Pour minimiser $\sum x_i y_i$, il faut associer les plus grandes valeurs d'un vecteur aux plus petites de l'autre.
- **Algorithme :** Trier $\mathbf{v}_1$ par ordre croissant et $\mathbf{v}_2$ par ordre décroissant, puis calculer le produit scalaire.
- **Preuve :** C'est une application directe de l'**inégalité de réarrangement** (rearrangement inequality) qui stipule que $\sum x_i y_i$ est minimisé quand les deux séquences sont triées en ordre opposé.
- **Complexité :** $O(n \log n)$ pour le tri.

2 Horaires de trains

Exercice 2 — Horaires de trains

Une ligne de train comporte deux gares, A et B . Des trains circulent de A vers B et de B vers A plusieurs fois par jour. Lorsqu'un train arrive à une gare, il a besoin d'un **temps de retournement** t (en minutes) avant de pouvoir repartir.

On dispose de l'horaire complet des trajets de la journée. Chaque trajet est défini par une heure de départ et une heure d'arrivée (au format HH:MM).

Objectif : Déterminer le nombre **minimum** de trains qui doivent commencer la journée en gare A et en gare B respectivement pour que tous les trajets de l'horaire puissent être assurés.

Règles :

- Un train arrivé en gare peut repartir après le temps de retournement t .
- Les trains ne peuvent pas effectuer de trajets hors de l'horaire.
- Un train arrivant à 12h00 avec $t = 5$ peut repartir à partir de 12h05.

Format d'entrée / sortie

Entrée : T cas de test. Chaque cas commence par t (temps de retournement), suivi de N_A (trajets $A \rightarrow B$) et N_B (trajets $B \rightarrow A$), puis les horaires correspondants.

Sortie : Pour chaque cas, **Case #i:** a b où a et b sont les nombres minimaux de trains démarrant en A et B .

- **Approche gloutonne** : Simuler chronologiquement les événements. Maintenir pour chaque gare le nombre de trains disponibles.
- Trier tous les événements (départs et arrivées) par ordre chronologique.
- À chaque départ depuis une gare, vérifier si un train est disponible ; sinon, en ajouter un nouveau.
- À chaque arrivée (+ temps t), incrémenter le stock de trains disponibles dans la gare d'arrivée.
- **Complexité** : $O(n \log n)$ pour le tri des événements.

3 Numbers — Puissances de $(3 + \sqrt{5})$

Exercice 3 — Numbers

On souhaite trouver les **trois derniers chiffres avant la virgule** du nombre $(3 + \sqrt{5})^n$.

Exemples :

- Pour $n = 5$: $(3 + \sqrt{5})^5 = 3935,73982\dots$ La réponse est 935.
- Pour $n = 2$: $(3 + \sqrt{5})^2 = 27,4164079\dots$ La réponse est 027.

Format d'entrée / sortie

Entrée : Un entier T suivi de T entiers n ($2 \leq n \leq 10^9$).

Sortie : Pour chaque cas, les trois derniers chiffres avant la virgule, avec zéros de tête si nécessaire.

- **Astuce algébrique** : Poser $\alpha = 3 + \sqrt{5}$ et $\beta = 3 - \sqrt{5}$. Remarquer que $0 < \beta < 1$.
- La suite $a_n = \alpha^n + \beta^n$ satisfait la **réurrence linéaire** :

$$a_n = 6a_{n-1} - 4a_{n-2}, \quad a_0 = 2, \quad a_1 = 6$$

car α et β sont racines de $x^2 - 6x + 4 = 0$.

- Puisque $0 < \beta^n < 1$, on a $\lfloor \alpha^n \rfloor = a_n - 1$. Donc les 3 derniers chiffres sont $(a_n - 1) \bmod 1000$.
- Pour n grand ($n \leq 10^9$), utiliser l'**exponentiation de matrice** :

$$\begin{pmatrix} a_n \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} 6 & -4 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} 6 \\ 2 \end{pmatrix}$$

Le calcul modulo 1000 avec exponentiation rapide donne une complexité $O(\log n)$.

4 Welcome to Code Jam

Exercice 4 — Welcome to Code Jam

Étant donné une chaîne de caractères S (composée de lettres minuscules et d'espaces), déterminer combien de fois la phrase "welcome to code jam" apparaît comme **sous-séquence** de S .

Rappel : Une sous-séquence est obtenue en sélectionnant des caractères de S à des positions *strictement croissantes* $s_0 < s_1 < \dots < s_{18}$, tels que la concaténation $S[s_0]S[s_1]\dots S[s_{18}]$ forme exactement "welcome to code jam" (19 caractères, espaces inclus).

Le résultat pouvant être très grand, on ne demande que les **4 derniers chiffres** (avec zéros de tête si nécessaire).

Format d'entrée / sortie

Entrée : Un entier N puis N lignes, chacune contenant une chaîne de lettres minuscules et d'espaces.

Sortie : Pour chaque cas, **Case #i:** XXXX où XXXX sont les 4 derniers chiffres du nombre de sous-séquences.

- **Programmation dynamique :** Soit $T = \text{"welcome to code jam"}$ de longueur $m = 19$.
- Définir $dp[j] =$ nombre de façons de former le préfixe $T[0..j-1]$ en utilisant les caractères vus jusqu'ici.
- Pour chaque caractère $S[i]$: pour chaque j de $m-1$ à 0, si $S[i] = T[j]$, alors :

$$dp[j] \leftarrow dp[j] + dp[j-1] \pmod{10000}$$
 avec la convention $dp[-1] = 1$ (la sous-séquence vide).
- **Complexité :** $O(|S| \times m)$ soit $O(19 \times |S|)$, linéaire en pratique.

5 Bassins versants (Watersheds)

Exercice 5 — Bassins versants

On vous donne une carte d'élévation sous forme d'un tableau 2D d'altitudes (H lignes, W colonnes). L'eau s'écoule selon les règles suivantes :

1. Depuis chaque cellule, l'eau coule vers **au plus un** de ses 4 voisins (N, O, E, S).
2. Si aucun voisin n'a une altitude inférieure, la cellule est un **puits** (*sink*).
3. Sinon, l'eau coule vers le voisin d'altitude minimale.

4. En cas d'égalité, on choisit le premier dans l'ordre : **Nord, Ouest, Est, Sud**.

Chaque cellule qui se déverse (directement ou indirectement) dans le même puits fait partie du même **bassin versant**. Étiqueter les bassins par des lettres minuscules de façon à ce que la concaténation ligne par ligne soit **lexicographiquement minimale**.

Format d'entrée / sortie

Entrée : T cartes. Chaque carte commence par H et W , suivis de H lignes de W entiers.

Sortie : Pour chaque cas, la carte étiquetée avec des lettres.

- **Simulation directe :** Pour chaque cellule, suivre le flux d'eau jusqu'à atteindre un puits. Deux cellules sont dans le même bassin si elles mènent au même puits.
- Utiliser un **Union-Find** (ou un simple parcours avec mémorisation) pour regrouper les cellules.
- Pour l'étiquetage lexicographique : parcourir les cellules de gauche à droite, de haut en bas, et attribuer la prochaine lettre disponible à chaque nouveau bassin rencontré.
- **Complexité :** $O(H \times W)$ avec mémorisation.

6 Langage extraterrestre

Exercice 6 — Langage extraterrestre

Des scientifiques ont découvert un langage extraterrestre composé de D mots, chacun de longueur exactement L . Les signaux reçus sont parfois ambigus : on reçoit un **motif** (*pattern*) de L *tokens*, où chaque token est :

- une lettre minuscule (certaine), ou
- un groupe de lettres entre parenthèses, par ex. (abc) , signifiant que la lettre réelle est l'une de celles du groupe.

Objectif : Pour chaque motif, déterminer combien de mots du dictionnaire correspondent.

Exemple : Avec le dictionnaire $\{\text{add}, \text{adc}, \text{bdd}, \text{bdc}\}$ et le motif $(ab)d(dc)$, les 4 mots correspondent.

Format d'entrée / sortie

Entrée : Trois entiers L, D, N . Puis D mots du dictionnaire, puis N motifs.

Sortie : Pour chaque motif, le nombre de mots correspondants.

- **Approche directe** : Pour chaque motif, parser les tokens. Pour chaque position i , construire l'ensemble des lettres possibles P_i .
- Pour chaque mot du dictionnaire, vérifier si $\forall i, \text{mot}[i] \in P_i$.
- **Optimisation possible** : Utiliser des masques de bits si l'alphabet est petit (≤ 26). Chaque ensemble P_i est un entier de 26 bits; la vérification est un simple AND.
- **Complexité** : $O(N \times D \times L)$, acceptable pour les contraintes données.

7 Nombres laids (Ugly Numbers)

Exercice 7 — Nombres laids

Un nombre est dit **laid** (*ugly*) s'il est divisible par au moins un des nombres premiers à un chiffre : 2, 3, 5 ou 7. Notez que 0 est laid, et que les nombres négatifs peuvent aussi l'être.

On vous donne une chaîne de chiffres, par exemple 123456. Entre chaque paire de chiffres adjacents, on peut insérer un signe +, un signe −, ou rien. Si la chaîne a D chiffres, il y a 3^{D-1} expressions possibles.

Objectif : Parmi toutes ces 3^{D-1} expressions, combien donnent un résultat **laid** ?

Exemples avec 123456 :

- $1 + 234 - 5 + 6 = 236$ est divisible par 2 $\Rightarrow$ laid.
- $123 + 4 - 56 = 71$ n'est divisible ni par 2, 3, 5, ni par 7 $\Rightarrow$ pas laid.

Format d'entrée / sortie

Entrée : Un entier T puis T chaînes de chiffres (longueur ≤ 13).

Sortie : Le nombre d'expressions évaluant à un nombre laid.

- **Inclusion-exclusion** : Un nombre est laid s'il est divisible par 2, 3, 5 ou 7. Compter les expressions dont le résultat est divisible par au moins un de ces premiers est plus facile par inclusion-exclusion sur les restes modulo $\text{lcm}(2, 3, 5, 7) = 210$.
- **Programmation dynamique** : $dp[i][r]$ = nombre d'expressions utilisant les i premiers chiffres donnant un résultat $\equiv r \pmod{210}$.
- À chaque position, on choisit de concaténer le chiffre suivant au nombre courant, ou d'insérer un + ou −.
- La réponse est 3^{D-1} moins le nombre d'expressions avec un résultat non divisible par 2, 3, 5 et 7 (i.e., $\text{gcd}(\text{résultat}, 210) = 1$).
- **Complexité** : $O(D^2 \times 210)$ (en regroupant les concaténations).

8 Optimisation du clavier T9

Exercice 8 — Placement optimal des lettres sur un clavier

Sur un ancien téléphone, les lettres sont réparties sur des touches. Pour taper une lettre, on appuie un certain nombre de fois sur sa touche : la 1 pression tape la 1 lettre, la 2 la 2, etc.

Objectif : Étant donné :

- K : le nombre de touches disponibles,
- P : le nombre maximal de lettres par touche,
- A : le nombre total de lettres dans l'alphabet (pas forcément 26),
- la fréquence de chaque lettre dans un message,

trouver le placement des lettres sur les touches qui **minimise** le nombre total de pressions pour taper le message.

Exemple : Sur le clavier standard, taper **snow** requiert 4 pressions pour **s** (touche 7, position 4) + 2 pour **n** + 3 pour **o** + 1 pour **w** = 10 pressions au total.

Format d'entrée / sortie

Entrée : T cas de test. Chaque cas contient K, P, A sur une ligne, puis A fréquences.

Sortie : Le nombre minimal de pressions pour chaque cas.

- **Approche gloutonne :** Trier les lettres par fréquence décroissante. Les lettres les plus fréquentes doivent être en position 1 (1 pression) sur les touches.
- Les K premières lettres (les plus fréquentes) coûtent 1 pression chacune.
- Les K suivantes coûtent 2 pressions, les K suivantes 3, etc., jusqu'à la position P .
- **Formule :** Si $f_1 \geq f_2 \geq \dots \geq f_A$ sont les fréquences triées :

$$\text{Coût} = \sum_{i=0}^{A-1} f_i \times \left(\left\lfloor \frac{i}{K} \right\rfloor + 1 \right)$$

- **Complexité :** $O(A \log A)$ pour le tri.

9 Sous-séquences croissantes (Increasing Speed Limits)

Exercice 9 — Sous-séquences strictement croissantes

On vous donne une séquence de n entiers. Calculez le nombre de **sous-séquences strictement croissantes** (non vides) de cette séquence.

Attention : Deux sous-séquences sont considérées différentes si elles utilisent des positions différentes, même si les valeurs sont identiques. La sous-séquence vide ne compte pas.

Exemple : Pour $(1, 4, 2, 3, 5, 5)$, la sous-séquence $(1, 2, 5)$ est comptée **deux fois** car le 5 peut provenir de deux positions différentes.

La réponse doit être donnée modulo un nombre premier donné.

Note : La séquence peut être très longue (jusqu'à 500 000 éléments) et est générée par un générateur pseudo-aléatoire défini par les paramètres n, m, X, Y, Z .

Format d'entrée / sortie

Entrée : T cas de test, chacun spécifiant les paramètres du générateur.

Sortie : Le nombre de sous-séquences strictement croissantes, modulo $2^{31} - 1$.

- **DP avec BIT/Fenwick :** Soit $dp[i]$ le nombre de sous-séquences croissantes se terminant en position i . On a :

$$dp[i] = 1 + \sum_{\substack{j < i \\ a_j < a_i}} dp[j]$$

- La somme naïve est en $O(n^2)$. Pour l'accélérer, utiliser un **arbre de Fenwick** (Binary Indexed Tree) indexé sur les valeurs compressées.
- Compresser les coordonnées : remplacer les valeurs par leur rang dans l'ensemble trié des valeurs distinctes.
- **Complexité :** $O(n \log n)$.
- **Réponse :** $\sum_{i=1}^n dp[i] \pmod{2^{31} - 1}$.

10 Mousetrap — Jeu de cartes

Exercice 10 — Mousetrap

Mousetrap est un jeu de cartes solitaire. On joue avec un paquet mélangé de K cartes numérotées de 1 à K , faces cachées.

Règles :

1. Révéler la carte du dessus et la placer en dessous du paquet.

2. Incrémenter un compteur (initialement à 1).
3. Si la valeur de la carte révélée **égale** le compteur : retirer la carte du jeu et remettre le compteur à 1.
4. Si le compteur atteint $K + 1$ sans retirer de carte, c'est perdu.
5. Si toutes les cartes sont retirées, c'est gagné.

Un paquet est dit **parfait** si on gagne **et** que les cartes sont retirées dans l'ordre croissant $(1, 2, 3, \dots, K)$.

Étant donné K et des indices de positions $d_1, d_2, \dots, d_n$, déterminer la valeur de la carte à chaque position d_i dans le paquet parfait de taille K .

Exemple : Pour $K = 4$, le paquet parfait est $(1, 4, 2, 3)$. On retire successivement les cartes 1, 2, 3, 4.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : K puis n indices $d_1, \dots, d_n$.

Sortie : Les valeurs aux positions demandées.

- **Simulation :** Construire le paquet parfait en simulant le jeu à l'envers : on sait que la carte i doit être retirée au i -ème retrait.
- Pour placer la carte i , il faut avancer de i positions dans le paquet restant (de manière circulaire).
- **Structure efficace :** Utiliser un **arbre de Fenwick** pour gérer les positions occupées et sauter rapidement k positions parmi les positions restantes.
- **Complexité :** $O(K \log^2 K)$ avec recherche binaire sur le BIT.

11 Crop Triangles — Triangles avec centroïde entier

Exercice 11 — Crop Triangles

On dispose de n points à coordonnées entières dans un plan. Un triangle est formé de 3 points **distincts**. Le **centroïde** d'un triangle de sommets (x_1, y_1) , (x_2, y_2) , (x_3, y_3) est :

$$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$$

Objectif : Compter le nombre de triangles dont le centroïde est un point à coordonnées **entières**. Les triangles d'aire nulle (points colinéaires) sont acceptés.

Les coordonnées des points sont générées par un générateur pseudo-aléatoire.

Format d'entrée / sortie**Entrée :** T cas de test. Chaque cas spécifie les paramètres du générateur de points.**Sortie :** Le nombre de triangles valides.

- **Condition nécessaire et suffisante :** Le centroïde est entier si et seulement si :

$$x_1 + x_2 + x_3 \equiv 0 \pmod{3} \quad \text{et} \quad y_1 + y_2 + y_3 \equiv 0 \pmod{3}$$

- **Classification :** Classer chaque point selon son couple de restes $(x_i \bmod 3, y_i \bmod 3)$. Il y a 9 classes possibles : $(0, 0), (0, 1), \dots, (2, 2)$.
- Soit $c_{(a,b)}$ le nombre de points dans la classe (a, b) . Il faut compter les triplets de classes $(a_1, b_1), (a_2, b_2), (a_3, b_3)$ tels que :

$$a_1 + a_2 + a_3 \equiv 0 \pmod{3} \quad \text{et} \quad b_1 + b_2 + b_3 \equiv 0 \pmod{3}$$

- Itérer sur les $\binom{9+2}{3}$ combinaisons de classes (avec répétitions) et compter les triangles correspondants en utilisant des coefficients multinomiaux.
- **Complexité :** $O(n + 9^3)$, essentiellement linéaire.

12 Traversée de la ville (Crossing the Road)

Exercice 12 — Traversée de la ville

Une ville est modélisée comme une grille de N lignes et M colonnes de blocs. Un piéton veut aller du coin **nord-est du bloc sud-ouest** au coin **sud-ouest du bloc nord-est** le plus rapidement possible.

Déplacements :

- Traverser une rue (1 bloc en N/S ou E/O) à un carrefour : **1 minute**, mais seulement si le feu est vert **pendant toute la traversée**.
- Longer un bloc (se déplacer le long d'un bord sans traverser) : **2 minutes**.
- Pas de déplacement en diagonale.

Feux de circulation : Le feu au carrefour i suit un cycle de période $S_i + W_i$:

- De T_i à $T_i + S_i$: vert pour la direction nord-sud, rouge pour est-ouest.
- De $T_i + S_i$ à $T_i + S_i + W_i$: vert pour est-ouest, rouge pour nord-sud.
- Le cycle se répète (et est aussi valide avant T_i).

Format d'entrée / sortie

Entrée : T cas. Chaque cas : N, M , puis pour chaque intersection ses paramètres S_i, W_i, T_i .

Sortie : Le temps minimum pour traverser la ville.

- **Modélisation en graphe :** Chaque coin de bloc est un nœud. Les arêtes représentent :
 - Le déplacement le long d'un bord (coût fixe de 2 minutes).
 - La traversée d'une rue à un carrefour (coût = temps d'attente + 1 minute).
- Le temps d'attente au carrefour dépend de l'instant d'arrivée t et de la direction voulue.
- **Algorithme de Dijkstra :** Comme les poids sont non négatifs (mais dépendent du temps), Dijkstra fonctionne directement. Le temps d'attente se calcule par modulo sur la période du feu.
- **Calcul du temps d'attente :** Pour une direction donnée à l'instant t au feu i :

$$\text{phase} = (t - T_i) \bmod (S_i + W_i)$$

Si la direction est verte et qu'il reste au moins 1 minute de vert, attente = 0 ; sinon, attendre le prochain passage au vert.

- **Complexité :** $O(NM \log(NM))$ avec Dijkstra et tas binaire.

13 Le Millionnaire

Exercice 13 — Le Millionnaire

Vous participez à un jeu télévisé. Avant le jeu, une roue de la fortune détermine P , la probabilité de gagner chaque pari. Vous démarrez avec X dollars et jouez M tours.

Règles :

- À chaque tour, vous pariez un montant quelconque b ($0 \leq b \leq$ fortune actuelle, pas limité aux centimes ou dollars entiers).
- Avec probabilité P , vous gagnez et votre fortune augmente de b .
- Avec probabilité $1 - P$, vous perdez b .
- Après M tours, vous gardez vos gains (arrondis à l'entier inférieur) uniquement si votre fortune est $\geq 1\,000\,000$ \$.

Objectif : En jouant de manière **optimale**, quelle est votre probabilité de finir millionnaire ?

Format d'entrée / sortie

Entrée : N cas. Chaque cas : M (entier), P (réel), X (entier). $1 \leq M \leq 15$, $0 \leq P \leq 1,0$, $1 \leq X \leq 1\,000\,000$.

Sortie : La probabilité maximale, avec au moins 6 décimales de précision.

- **Discrétisation :** Après M tours, la fortune est un multiple de $1\,000\,000/2^M$. On peut discrétiser la fortune en $2^M + 1$ niveaux.
- **DP descendante :** Soit $dp[r][k]$ la probabilité maximale d'atteindre 1M\$ en r tours restants, sachant qu'on est au k -ième niveau.
- **État final :** $dp[0][k] = 1$ si $k \geq 2^M$ (fortune ≥ 1 M\$), 0 sinon.
- **Transition :** Pour chaque niveau k et mise b (en unités de niveaux) :

$$dp[r][k] = \max_{0 \leq b \leq k} \left\{ P \cdot dp[r-1][\min(k+b, 2^M)] + (1-P) \cdot dp[r-1][k-b] \right\}$$
- **Propriété de concavité :** $dp[r][\cdot]$ est concave, donc le maximum est atteint en cherchant parmi quelques candidats (pas besoin de tester tous les b).
- **Complexité :** $O(M \cdot 2^M)$, faisable car $M \leq 15$.

14 Raquette anti-mouche (Fly Swatter)

Exercice 14 — Raquette anti-mouche

Quelle est la probabilité de toucher une mouche avec une raquette de tennis ?

Modèle :

- La raquette est un anneau de rayon extérieur R et d'épaisseur t (rayon intérieur $R-t$).
- L'anneau est couvert de cordes horizontales et verticales, symétriques par rapport au centre. Chaque corde est un cylindre de rayon r . L'espacement entre cordes (gap) est g .
- La mouche est une sphère de rayon f .
- La raquette se déplace perpendiculairement au plan des cordes.
- Le centre de la mouche est uniformément distribué dans le disque de rayon R .

Un **contact** a lieu si la mouche touche l'anneau ou une corde. Calculer la probabilité de contact.

Format d'entrée / sortie**Entrée :** T cas. Chaque cas : f, R, t, r, g (réels).**Sortie :** La probabilité de toucher la mouche, avec au moins 6 décimales de précision.

- **Complémentaire :** Calculer l'aire où la mouche **passé à travers** (pas de contact), puis $P(\text{contact}) = 1 - \frac{\text{aire de passage}}{\pi R^2}$.
- La mouche passe à travers un trou rectangulaire entre les cordes seulement si son centre est à distance $> f$ de chaque corde et de l'anneau.
- **Symétrie :** Exploiter la symétrie par rapport aux axes. Il suffit de calculer dans un quart du disque.
- Pour chaque cellule rectangulaire entre cordes, calculer l'aire effective (intersection de la cellule réduite de $f + r$ de chaque côté avec le disque intérieur de rayon $R - t - f$). Attention aux coins arrondis des cellules.
- L'aire de passage dans une cellule nécessite de calculer l'intersection d'un rectangle arrondi avec un disque.
- **Complexité :** Proportionnelle au nombre de cellules dans un quart de la raquette.

15 Peindre une clôture

Exercice 15 — Peindre une clôture

Une clôture est composée de 10 000 sections contiguës, numérotées de 1 à 10 000. Vous recevez des offres de peintres. Chaque peintre propose de peindre un **intervalle contigu** de sections dans une couleur donnée.

Contraintes :

1. Chaque section doit être peinte (couverture complète).
2. Au plus **3 couleurs** sont utilisées au total.

Objectif : Si les deux contraintes peuvent être satisfaites, trouver le nombre **minimum** d'offres à accepter. Sinon, indiquer que c'est impossible.

Format d'entrée / sortie**Entrée :** T cas. Chaque cas : N offres, puis pour chaque offre : couleur, début, fin.**Sortie :** Le nombre minimum d'offres, ou IMPOSSIBLE.

- **Observation :** Il n'y a que $\binom{C}{3}$ combinaisons possibles de 3 couleurs (où C est le nombre de couleurs distinctes parmi les offres). Pour chaque combinaison :

- **Sous-problème** : Avec un sous-ensemble de couleurs fixé, couvrir $[1, 10000]$ avec le minimum d'intervalles choisis parmi les offres de ces couleurs.
- C'est le problème classique de **couverture d'intervalle** (*interval covering*), résolvable par un algorithme glouton en $O(N \log N)$: toujours choisir l'intervalle qui va le plus loin parmi ceux qui commencent avant ou au niveau de la fin de la couverture actuelle.
- Aussi vérifier les cas avec 1 ou 2 couleurs.
- **Complexité** : $O(C^3 \times N \log N)$ dans le pire cas.

16 Apocalypse Soon — Survie sur grille

Exercice 16 — Apocalypse Soon

Le monde est une grille de $R \times C$ nations. Chaque nation a une force S et attaque chaque jour son voisin le plus fort (parmi N, S, E, O), en cas d'égalité en choisissant le plus au nord puis le plus à l'ouest. L'attaque réduit la force du voisin de S . Une nation tombe si sa force atteint 0.

Votre particularité : Vous contrôlez la nation en position (c, r) et pouvez choisir librement quel voisin attaquer (ou ne rien faire) chaque jour. Toutes les attaques sont simultanées.

Objectif : Maximiser le nombre de jours de survie. Si vous pouvez survivre indéfiniment (tous les voisins sont détruits), afficher "forever".

Format d'entrée / sortie

Entrée : T cas. Chaque cas : C, R, c, r , puis la grille de forces.

Sortie : Le nombre maximal de jours, ou "forever".

- **Simulation** : La grille est petite. Simuler jour par jour en testant les différentes stratégies pour votre nation.
- **Recherche exhaustive** : À chaque jour, vous avez au plus 5 choix (attaquer un voisin ou ne rien faire). Utiliser une recherche avec élagage (*branch and bound*).
- **Condition de fin** : Si tous vos voisins sont détruits et votre force est > 0 , vous survivez indéfiniment.
- Attention : les attaques sont **simultanées** — utiliser les forces du début du tour pour tous les calculs d'attaque.
- **Heuristique** : Prioriser l'élimination des voisins qui vous attaquent le plus fortement.

17 Bols mélangeurs (Mixing Bowls)

Exercice 17 — Bols mélangeurs

Vous suivez une recette pour préparer un plat. La recette est un arbre de mélanges :

- Chaque mélange est fait en combinant des **ingrédients** dans un bol vide.
- Un ingrédient est soit un ingrédient de base (déjà disponible), soit un autre mélange (à préparer au préalable dans son propre bol).
- On ne peut **pas** ajouter d'ingrédients à un mélange déjà fait ; il faut un bol vide.
- Après avoir versé un mélange comme ingrédient dans un autre bol, le bol source est libéré et réutilisable.

Objectif : Déterminer le nombre **minimum** de bols nécessaires, en choisissant l'ordre optimal de préparation des mélanges.

Format d'entrée / sortie

Entrée : T cas. Chaque cas décrit l'arbre de la recette (nœuds = mélanges, feuilles = ingrédients de base).

Sortie : Le nombre minimal de bols.

- **Modélisation en arbre :** La recette forme un arbre enraciné. Chaque nœud interne est un mélange, chaque feuille est un ingrédient de base.
- **Stratégie optimale :** C'est analogue au problème du **nombre de registres** pour évaluer un arbre d'expressions. La solution classique est l'algorithme d'**Ershov/Sethi-Ullman**.
- Si un nœud a des enfants nécessitant $b_1 \geq b_2 \geq \dots \geq b_k$ bols respectivement, le nombre de bols pour ce nœud est :

$$\max(b_1, b_2 + 1, b_3 + 2, \dots, b_k + (k - 1))$$

(en traitant d'abord l'enfant le plus coûteux).

- Les ingrédients de base ne nécessitent pas de bol (ils sont directement disponibles), donc seuls les enfants qui sont eux-mêmes des mélanges comptent.
- **Complexité :** $O(n \log n)$ pour chaque cas (tri des enfants à chaque nœud).

18 Groupement par facteurs premiers

Exercice 18 — Groupement par facteurs premiers communs

On considère une suite d'entiers consécutifs de A à B (inclus). Initialement, chaque entier est dans son propre ensemble.

Pour chaque paire d'entiers dans l'intervalle $[A, B]$, si ces deux entiers partagent un **facteur premier** $\geq P$, on fusionne leurs deux ensembles.

Objectif : Déterminer le nombre d'ensembles distincts à la fin du processus.

Format d'entrée / sortie

Entrée : C cas de test. Chaque cas : trois entiers A, B, P . ($1 \leq A \leq B \leq 10^6$, $2 \leq P$).

Sortie : Pour chaque cas, le nombre d'ensembles distincts.

- **Union-Find** : Initialiser chaque entier dans son propre ensemble.
- Pour chaque nombre premier $p \geq P$ (jusqu'à B), fusionner tous les multiples de p dans $[A, B]$ entre eux.
- **Crible** : Utiliser un crible d'Ératosthène pour énumérer les premiers $\geq P$ jusqu'à B .
- Pour chaque premier p , le premier multiple $\geq A$ est $\lceil A/p \rceil \times p$. Fusionner ce multiple avec les suivants $(+p)$ tant qu'on reste $\leq B$.
- **Complexité** : $O((B - A) \log \log B)$ pour le crible + quasi-linéaire pour les fusions avec union par rang et compression de chemin.
- La réponse est le nombre de composantes connexes dans le Union-Find.

Tableau récapitulatif des techniques

Ex.	Problème	Techniques principales
1	Produit scalaire minimal	Tri, inégalité de réarrangement
2	Horaires de trains	Simulation gloutonne, tri d'événements
3	Numbers $(3 + \sqrt{5})^n$	Récurrence linéaire, exponentiation matricielle
4	Welcome to Code Jam	Programmation dynamique sur sous-séquences
5	Bassins versants	Parcours de graphe, Union-Find, BFS/DFS
6	Langage extraterrestre	Parsing, ensembles de caractères, masques de bits
7	Nombres laids	DP modulo 210, inclusion-exclusion
8	Clavier T9	Tri glouton par fréquence
9	Sous-séquences croissantes	DP + arbre de Fenwick (BIT)
10	Mousetrap	Simulation, BIT pour positions circulaires
11	Crop Triangles	Comptage modulo 3, combinatoire
12	Traversée de la ville	Dijkstra, graphe pondéré dynamique
13	Le Millionnaire	DP sur fortune discrétisée, concavité
14	Raquette anti-mouche	Géométrie computationnelle, intégration
15	Peindre une clôture	Couverture d'intervalles, glouton
16	Apocalypse Soon	Simulation, recherche exhaustive
17	Bols mélangeurs	Arbre, évaluation optimale (Sethi-Ullman)
18	Groupement par facteurs	Union-Find, crible d'Ératosthène

Bon courage et bonne programmation !